

ZERO-DAY DISCOVERY FRAMEWORK

Contents

Introduction 2

Zero-day Discovery Framework Main Section 3

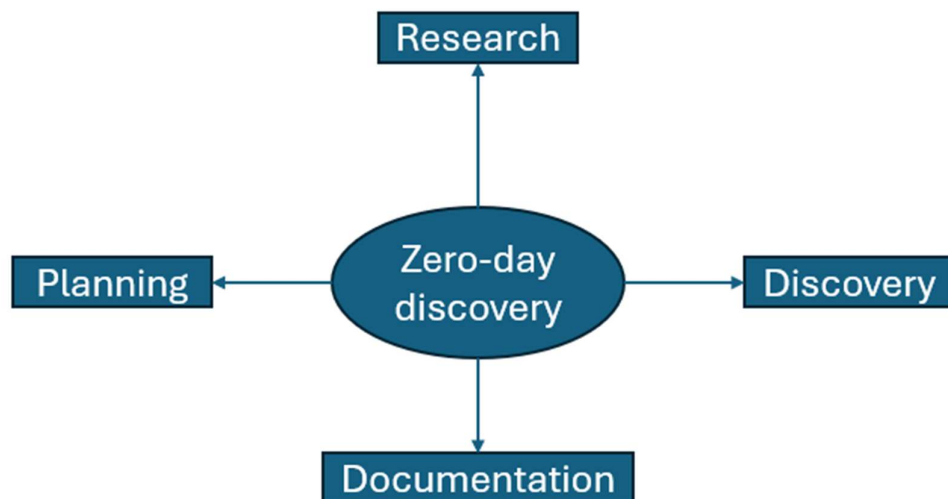
Zero-day discovery Workflow 4

Zero-day Discovery Framework Trade-Offs..... 5

Introduction

This zero-day discovery framework is designed to help security teams discover zero-day vulnerabilities in their system. It is also designed to help red team activities in terms of zero-day usage. This zero-day discovery framework was developed as zero-days are currently the biggest and most difficult threat to stop in cybersecurity. It was also developed to provide zero-day researchers with a structured and simple approach to discovering zero-days. This zero-day discovery framework will cover the main discovery structure, workflow and trade-offs.

This zero-day discovery framework cannot promise perfect discovery in systems. It also cannot promise provide patches or fixes once the zero-day has been discovered. However, this framework can promise to improve the efficiency and possibly increase discovery rates of zero-day vulnerabilities. It is important to remember that these zero-discovery framework isn't made to help attackers discover vulnerabilities. Make sure to adapt the steps and actions as needed.



The spider diagram above shows the 4 main steps included in this zero-day discovery framework. This diagram does not show the order of the steps or all steps involved in framework. You can find more details in the sections below.

Zero-day Discovery Framework Main Section

- This zero-day discovery framework uses 5 steps to discover and to store zero-days securely.
- The first step is planning.
- This is where you set out the scope, budget and target for the zero-day discovery.
- This can be done by asking questions like what is the system? What is the main objective? What resources and budget are available?
- This step is important to ensure that researchers or individuals understand what their target is and their available resources.
- The second step is research.
- This is where you study the system you plan to discover a zero-day vulnerability in.
- This can be done by looking at system creation processes, authentication processes and the purpose of the system
- This step is important as you cannot discover vulnerabilities in a system you do not understand.
- The third step is discovery.
- This is where you discover the zero-day vulnerability in the target system
- This can be done by reviewing open-source code, using AI-driven vulnerability discovery and looking at the sections of the system that weren't built with security in mind.
- This step is important as it allows you to discover the zero-day vulnerability by looking at the components that no one thinks would be exploited.
- The fourth step is documentation.
- This is where you document the zero-day vulnerability discovery in a clear format.
- This can be done using the structure What (What was the vulnerability?) → Why (Why does it exist?) → How (How was it discovered?) → Threat model (Who could exploit this vulnerability?).
- Red teams may use suggested use case or target case in the documentation structure for testing security systems.
- This step is important as it allows you to document the vulnerability in structured format which can later be used in a red team activity or for patching.
- The fifth step is secure storage
- This step is where you securely store the documentation to ensure that no one else can find or use the zero-day vulnerability.
- This can be done by storing the documentation in encrypted container storage and using OPSEC and RBAC for information control.
- This step is extremely important to ensure that the zero-day can only be used by the party that discovered it.
- You should also make sure to verify before documentation occurs.

Zero-day discovery Workflow

- First, the user finds the zero-day discovery framework via GitHub while searching through cybersecurity content.
- Second, the user plans to discover a zero-day in current authentication app MFA.
- They want a zero-day vulnerability that allows them to view TOTP codes and secret keys.
- The user has large technical knowledge and has the resources of an organised cybercriminals group.
- Third, the user begins to study how authentication app MFA works with authentication and security.
- The user reads about the standard, how the secret key is used to generate secure TOTP codes and to verify to the server.
- The user also understands that MFA traffic is encrypted by HTTPS so they cannot simply see the secret key or codes.
- The user also figures out that MFA authentication is handled on a specific port of the server.
- Fourth, the user starts the discovery process by looking at the connection between the user's device and the server.
- The user figures out that although the traffic is encrypted, the metadata from the packets is still shown.
- The user uses their own account to collect the packets using a packet sniffer like Wireshark to monitor their own traffic to the server.
- The user finds out from the metadata the server's MFA port and IP address even if it is using a reserve proxy.
- The user realises that if they can set up a real-time MFA proxy that uses the MFA port and mimics the IP address of the server then they may be able to see secret keys and TOTP codes.
- They continue by noticing that they can also redirect the traffic back to the real server to avoid user suspicion
- Fifth, the user documents the zero-day vulnerability using what → why → how → suggested use case structure.
- The user calls the zero-day vulnerability MFA redirection attack, they explain that the metadata can be used to extract needed information, which can then be used to set up a proxy server, which then can get the TOTP and secret keys then redirect the user to the real server making it seem like a connection issue.
- The user documents that they discovered it by looking at the connection between the user's device and the server and suggested to use it to compromise high value accounts using authentication app MFA.
- Sixth, the user stores the documentation securely.
- The user uses an VeraCrypt encrypted container with a 24-character password, multiple key files and the documentation is 7-zip encrypted with a 16-character password.

Zero-day Discovery Framework Trade-Offs

- **This zero-day discovery framework also has trade-offs just like any other system I've created.**
- **I have identified 6 main trade-offs with this framework.**
- **The first trade-off is cost.**
- **This trade-off exists as it can be expensive to discover zero-day vulnerabilities due to the tooling, system analysis and even sometimes infrastructure needed.**
- **This trade-off can be managed by using existing free tools, allocating budget and focusing one component of the system instead of in general.**
- **The second trade-off is dual use.**
- **This trade-off exists as it can be used by both defenders and attackers to find zero-day vulnerabilities in systems.**
- **Defenders can use it to discover and then patch zero-day vulnerabilities within their systems.**
- **Attackers can use it to discover and then exploit zero-day vulnerabilities within target systems.**
- **This trade-off can be managed by secure storage and using a defence in approach that forces an attacker to use multiple zero-days.**
- **The third trade-off is patch difficulty.**
- **This trade-off exists as patching certain zero-days especially the MFA protocol zero-day can be extremely difficult and complex to patch.**
- **This trade-off can be managed by using AI-driven assistance, patching teams and clearly warning users of the vulnerability.**
- **The fourth trade-off is cognitive load.**
- **This trade-off exists as zero-day discovery requires focus, dedication, energy and the willingness to study a system.**
- **This means that analysts and researchers can be exhausted and burned out if not managed carefully.**
- **This trade-off can be managed by rotating schedules, incremental progress and clear and planned discovery structure.**
- **The fifth trade-off is time consumption**
- **This trade-off exists as zero-day discovery can require months or even years to complete successfully.**
- **This trade-off can be managed by using timescales and using efficient zero-day discovery techniques.**
- **The sixth trade-off is one-time use.**
- **This trade-off exists as zero-day vulnerabilities can only be used once before they are burned and patched by a defender.**
- **This trade-off can be managed by using the zero-day vulnerability for an specific target that has a high ROI.**